

Configurable Methylation Pipeline — Architecture

Portal, workflow database, middle-tier, workers, and scoped variables

2026-06-09

Table of contents

1	Presentation map (four layers)	2
2	1. Project configuration	3
2.1	Top-level structure	3
2.2	Cohort hierarchy	4
2.3	Comparisons	4
2.4	Example snippet (two-group OvR style)	4
3	2. JSON Schema hierarchy	5
3.1	Reference tree	5
3.2	\$defs in project_config.schema.json	5
3.3	Per-step schemas (schemas/config/)	6
3.4	Constraint enforcement	7
4	3. Portal config editor	7
4.1	Editor flow	7
4.2	From project JSON to workflow run	8
5	4. Workflow database model	8
5.1	Tables by layer	8
5.2	Control-flow node types	9
5.3	Entity relationship (simplified)	9
5.4	Scoped variables — declaration, assignment, and use	10
5.4.1	Storage model	10
5.4.2	Declaration (definition-time)	11
5.4.3	Assignment (runtime)	11
5.4.4	Use (read-path)	12
5.5	Worker-driven conditions and branching	12
5.6	Monte Carlo runs and stratified partitions	13
5.6.1	Workflow engine bridge	13
5.6.2	Example workflow topology (MethylValidationFlow)	14
5.7	Cluster metadata in DB	14

6	5. Two-group example: config → workflow tree	15
6.1	5.1 Project intent (PCa3-style)	15
6.2	5.2 Instance <code>context_json</code> (runtime)	15
6.3	5.3 Workflow tree (<code>DataDrivenPipeline</code>)	16
6.4	5.4 Per-chromosome dependency (one comparison)	16
6.5	5.5 Execution scale (PCa OvR example)	16
6.6	5.6 Input templates (resolved at claim time)	17
7	6. Worker protocol and middle-tier	17
7.1	REST endpoints	17
7.2	Request / submit sequence	17
7.3	Middle-tier responsibilities	17
7.4	Capability matching	18
8	7. Cluster and shared storage model	18
8.1	Assumptions	19
8.2	Architecture diagram	19
8.3	Minimizing total processing time	19
8.4	Portal + planner + engine (end-to-end)	20
9	Further reading	20
10	Rendering this document	20

Presentation-oriented overview of how a **project configuration** (JSON), a **schema-validated portal editor**, a **workflow database** (including **scoped variables**), a **stateless middle-tier**, and **remote workers** cooperate to run a configurable methylation pipeline on a shared-storage cluster.

Audience: engineers and stakeholders reviewing the company portal, backend database, middle-tier, and compute workers.

Related code: [schemas/config/](#), [tools/methyl-config-editor/](#), [workflow_engine/](#)

1 Presentation map (four layers)

Layer	Technology	Responsibility
Portal	uniGUI web app (<code>MethylConfigEditorWeb</code>)	Edit <code>project.json</code> against JSON Schema; start runs
Database	Azure SQL or Azure PostgreSQL (<code>wf</code> schema)	Store workflow tree, instances, executions, leases
Middle-tier	Delphi <code>WfEngine</code> + REST (<code>/v1/*</code>)	Stateless HTTP proxy; invokes engine stored procedures
Workers	Python/CLI on cluster nodes	Poll tasks by capability; read/write shared paths

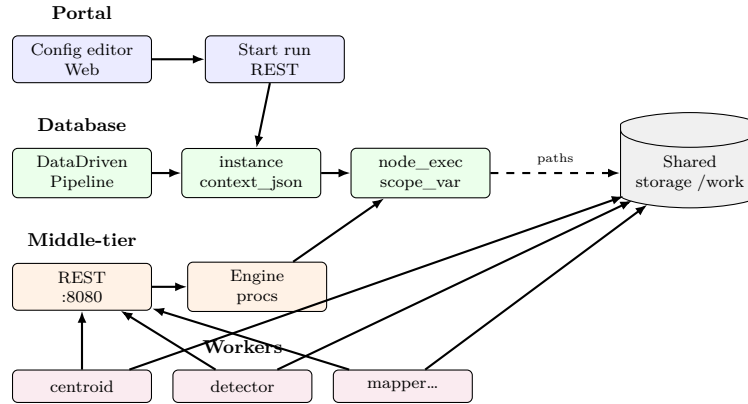


Figure 1: Four-layer architecture: portal, database, middle-tier, and cluster workers.

2 1. Project configuration

A **project config** is a single JSON file that describes *what* to run: cohorts, comparisons, chromosomes, contexts, and per-step tool parameters. It is the scientific contract for one methylation study.

Canonical schema: [schemas/config/project_config.schema.json](#)

Pydantic model: [packages/methylutils/methyl_utils/pipeline_config.py](#)

In-repo examples: [tools/methyl-config-editor/configs/](#)

2.1 Top-level structure

Only **project_name** and **output_base** are required. Everything else is optional and validated by schema.

Field	Role
<code>project_name</code>	Identifier; output lives under <code>{output_base}/{project_name}/</code>
<code>output_base</code>	Global output root (e.g. <code>/work/prostate-cancer</code>)
<code>controls / disease comparisons</code>	Two-sided layout: control vs disease cohorts Shorthand string or explicit list of control vs disease pairs
<code>chromosomes</code>	Shared list ("1"..."22", "X", "Y")
<code>contexts</code>	Methylation contexts: CG, CHG, CHH
<code>path_remap</code>	Prefix replacement when sample paths moved (e.g. NAS → cluster mount)

Field	Role
<code>step_config</code>	Per-step overrides: <code>centroid</code> , <code>detection</code> , <code>mapper</code> , <code>enricher</code> , <code>classifier</code> , <code>predictor</code> , <code>validation</code> , <code>progression</code> , ...

2.2 Cohort hierarchy

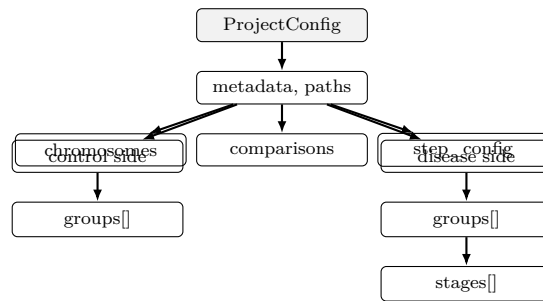


Figure 2: Project configuration JSON hierarchy.

Control side (controls): one or more groups, e.g. `all` with a CSV of healthy sample paths.

Disease side (diseases): groups may nest **stages** (e.g. Gleason PCa1–PCa4 under parent PCa). Resolved centroid labels become `{parent}_{stage}` → `PCa_PCa1`.

2.3 Comparisons

comparisons can be:

Form	Behavior
<code>"control_vs_each_disease"</code>	First control group vs each resolved disease leaf
<code>"all_pairs"</code>	Every control leaf × every disease leaf
<code>[{control_group,</code> <code>disease_group,</code> <code>comparison_label?}, ...]</code>	Explicit list

Detection output paths follow: `detections/{control_group}/{disease_group}/`.

2.4 Example snippet (two-group OvR style)

External reference project (not in repo): `project_PCa3.json` with one control (`all`) and two disease stages (`PCa_Low`, `PCa_High`). In-repo analogue with four stages:

```
{
  "project_name": "Healthy_vs_PCa1-4-CG",
  "output_base": "/work/prostate-cancer",
  "controls": {
    "label": "healthy",
    "groups": [{ "label": "all", "sample_paths": ["/work/prostate-cancer/data/healthy.csv"] }]
  },
  "diseases": {
    "label": "cancer",
    "groups": [{
      "label": "PCa",
      "stages": [
        { "label": "PCa1", "description": "Gleason Score 3+3", "sample_paths": [...] },
        { "label": "PCa2", "description": "Gleason Score 3+4", "sample_paths": [...] }
      ]
    }]
  },
  "comparisons": "control_vs_each_disease",
  "chromosomes": ["1", "2", "...", "22", "X", "Y"],
  "contexts": ["CG"],
  "step_config": {
    "centroid": { "base_config": { "use_gpu": true, "min_coverage": 4 } },
    "detection": { "alpha": 0.05 },
    "progression": { "ordered_comparison_labels": ["PCa1", "PCa2", "PCa3", "PCa4"] }
  }
}
```

For a **two-group OvR** study (one control vs two disease stages), use "comparisons": "control_vs_each_disease" with two disease leaves (PCa_Low, PCa_High) — see Section 5.

3 2. JSON Schema hierarchy

Schemas are **JSON Schema draft 2020-12**. The top-level document is generated from Pydantic (ProjectConfig) and kept in sync via `methyl-export-config-schemas` ([packages/methylvalidation/methyl_validation/config_schema_registry.py](#)).

3.1 Reference tree

3.2 \$defs in project_config.schema.json

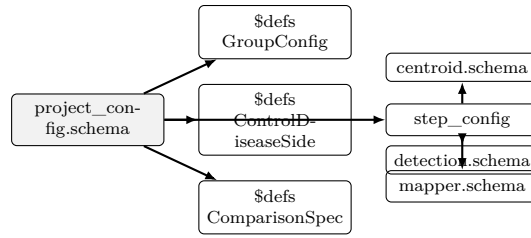


Figure 3: JSON Schema reference tree (excerpt).

Definition	Required fields	Purpose
ComparisonSpec	control_group, disease_group	One pairwise comparison
ControlDiseaseSide	label	Control or disease side with groups []
GroupConfig	label	Cohort or stage; optional stages [], subcluster , level_labels_path
SubclusterRequest	(none)	Optional MethylCluster pre-centroid clustering

3.3 Per-step schemas (schemas/config/)

File	Step
centroid.schema.json	MethylCentroid
detection.schema.json	MethylDetector
mapper.schema.json	MethylMapper
enricher.schema.json	MethylEnricher
classifier.schema.json	MethylClassifier
predictor.schema.json	MethylPredictor
validation.schema.json	MethylValidation (+ nested validation_monte_carlo, validation_regulatory, validation_backend_profiles, ...)
progression.schema.json	MethylDiseaseProgression
alignment_qc.schema.json	Alignment QC

step_config in the project file is an open object: keys name the step; values are merged into each tool's config at runtime (CLI and workers still override with flags/files).

3.4 Constraint enforcement

- **Portal editor:** loads `*.schema.json` and blocks invalid types, missing required fields, and bad enums at edit time (Section 3).
- **Python runtime:** `ProjectConfig.model_validate()` on `load_project()` enforces the same rules before any pipeline step runs.
- **Comparisons resolution:** `get_comparisons()` in `pipeline_config.py` expands short-hands and validates that `control_group` / `disease_group` labels exist in resolved cohort leaves.

4 3. Portal config editor

There is **no separate React/TypeScript portal** in this repository. JSON editing is provided by **methy1-config-editor**: a schema-driven Delphi application with a **web port** suitable for embedding in a company portal.

Component	Path
Desktop (VCL)	<code>tools/methy1-config-editor/Methy1ConfigEditor.dproj</code>
Web (uniGUI)	<code>tools/methy1-config-editor/Methy1ConfigEditorWeb.dproj</code> — default port 8077
Schema loader	<code>src/Schema/JsonSchemaLoader.pas</code>
Web property grid	<code>src/Web/SchemaPropertyGrid.pas</code>

4.1 Editor flow

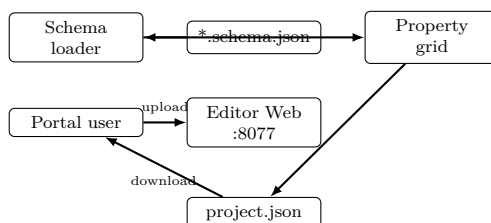


Figure 4: Schema-driven portal config editor flow.

Behavior:

1. Administrator points the web app at `schemas/config` (`Paths.SchemasRoot` in `methy1-config-editor-web.ini`).
2. User selects schema `project_config.schema.json`, uploads or creates JSON.
3. **Scalars** edit inline in `TUniPropertyGrid`; **objects/arrays** open modal nested editors.
4. `$ref` to sibling files (e.g. `centroid.schema.json`) resolves at load time — no code generation.
5. User downloads validated JSON.

4.2 From project JSON to workflow run

The portal (or a separate **planner service**) maps a validated project config into a **workflow_instance.context_json** — runtime variables for the workflow engine, not a copy of the full project file.

Typical mapping:

Project config	context_json
Resolved comparisons	comparisons[] with label, centroid2Dir, detectOutDir, ...
chromosomes	chromosomes[]
output_base + paths	centroid1Dir, projectPath, tool name strings
step_config.progression.ordered_comparisons	orderedComparisons
labels	ComparisonLabels

Example instance payload: [workflow_engine/sql/instance_context_examples/pca_ovr.json](#).

Starting a run (middle-tier):

```
POST /v1/workflows/instances
{ "workflow_version_id": <DataDrivenPipeline version id>, "context_json": { ... } }
```

5 4. Workflow database model

The workflow engine stores **definitions** (reusable trees) and **runtime** (instances, executions, leases). The same logical model deploys to **Azure SQL Database** (T-SQL, json columns) or **Azure PostgreSQL** (jsonb). Stored procedure names are contractually identical ([workflow_engine/contract/db_objects.yaml](#)).

Deploy scripts:

- SQL Server: bundled [workflow_engine/sql/MethylPipeline.sql](#) + parity scripts
- PostgreSQL: [workflow_engine/sql_pg/](#) (00_schema.sql ... 07_scope_encoding_parity.sql)

5.1 Tables by layer

Layer	Tables
Definition	workflow_def, workflow_version, workflow_node, workflow_edge, workflow_action, workflow_input_template, workflow_input_binding, variable_output_binding, node_scope_default

Layer	Tables
Runtime	<code>workflow_instance</code> , <code>node_execution</code> , <code>task_lease</code> , <code>loop_state</code> , <code>execution_context</code> , <code>scope_variable</code> , <code>instance_cursor</code>
Workers / cluster	<code>worker</code> , <code>worker_token</code> , <code>cluster</code>

5.2 Control-flow node types

`workflow_node.node_type` CHECK constraint:

ACTION | SEQUENCE | PARALLEL | IF | SWITCH | REPEAT | WHILE | FOREACH

Type	Role
ACTION	Leaf task; bound to <code>workflow_action</code> ; becomes READY for workers
SEQUENCE	Run children in <code>child_order</code>
PARALLEL	Run all children concurrently; complete when all succeed
FOREACH	Iterate a JSON array from <code>context_json</code> (<code>foreach_collection_var</code>); <code>foreach_parallel=1</code> fans out all iterations at once
IF / SWITCH	Branch on scope variable or prior task result
REPEAT / WHILE	Fixed-count or condition loops

`workflow_edge.branch_kind`: SEQUENCE, PARALLEL, THEN, ELSE, CASE, DEFAULT, BODY.

5.3 Entity relationship (simplified)

```

workflow_def      1-* workflow_version
workflow_version 1-* workflow_node, 1-* workflow_instance
workflow_node    *-* workflow_edge; 0..1 workflow_input_template
workflow_instance 1-* node_execution
node_execution    0..1 task_lease; scope via scope_variable
cluster           1-* worker → worker_token

```

Figure 5: Workflow database entity relationships (simplified).

5.4 Scoped variables — declaration, assignment, and use

Scoped variables are a first-class workflow feature: they carry **instance parameters**, **loop indices**, **FOREACH item fields**, and **outputs from prior worker actions** through the execution tree. The engine resolves them when building `input_json` and when choosing IF / SWITCH / WHILE branches.

Why they exist:

1. **Complex control flow** — conditions that are too rich for a static workflow definition (QC gates, pagination, model-quality thresholds, custom business rules) can be evaluated inside a worker ACTION; the worker writes an integer or JSON fragment back into scope, and downstream IF / SWITCH / WHILE nodes read that value.
2. **Monte Carlo validation** — each iteration needs a **fresh stratified train/validation partition** of cohort samples (e.g. 80% train / 20% test per cohort). Those partition descriptors are stored as scoped variables (`mc.taskConfig`) and injected into every pipeline step for that iteration.

5.4.1 Storage model

Object	Layer	Role
<code>wf.scope_variable</code>	Runtime	(<code>workflow_instance_id</code> , <code>scope_node_execution_id</code> , <code>var_name</code>) → <code>value_json</code>
<code>wf.node_scope_default</code>	Definition	Variables declared when a composite scope opens (<code>var_name</code> , <code>default_expr</code>)
<code>wf.variable_output_binding</code>	Definition	Maps ACTION output → scope variable (<code>result_code</code> or <code>output_path</code> into <code>output_json</code>)
<code>workflow_node.condition_var</code> / <code>switch_var</code>	Definition	IF / SWITCH / WHILE read these names from scope

Scope identity: `scope_node_execution_id = 0` is the **instance root** (global scope). Each composite node (SEQUENCE, PARALLEL, REPEAT, WHILE, FOREACH, IF, SWITCH) opens a child scope tied to its `node_execution.id`. Lookup walks **up the parent chain** until a name is found (`wf.wf_get_scope_variable_json`).

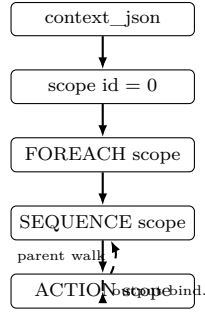


Figure 6: Scoped variable hierarchy along the execution tree.

5.4.2 Declaration (definition-time)

node_scope_default declares variables that appear when a node’s scope opens. Values are **expressions**, not literals: the engine resolves `${...}` placeholders against the current scope before assignment.

Example from per-comparison nodes in legacy PCa seeds: `centroid2Dir`, `detectOutDir`, and `comparisonLabel` are seeded as defaults on each comparison subtree so the workflow definition stays generic while paths vary by comparison label.

5.4.3 Assignment (runtime)

Mechanism	When	Example
Instance bootstrap	StartInstance	Every top-level key in <code>context_json</code> → scope 0
Scope open	Composite activation	Copy parent scope + apply <code>node_scope_default</code>
FOREACH iteration	Each loop body activation	Bind <code>foreach_item_var</code> fields (<code>label</code> , <code>chromosome</code> , ...) and <code>foreach_index_var</code>
REPEAT / WHILE	Loop body	<code>\${ctx.iterationNo}</code> in <code>execution_context</code>
ACTION complete	Worker submit	<code>variable_output_binding: result_code</code> or JSON path from <code>output_json</code>
Monte Carlo planner	Instance start / iteration	<code>mc.taskConfig</code> , <code>mc.runId</code> , <code>mc.phase</code> , ...

Engine write-path: `wf_sql_scope_writepath_parity.sql` (`wf_set_scope_variable`, `wf_open_scope`, `wf_apply_output_bindings`).

5.4.4 Use (read-path)

Input templates and bindings resolve placeholders at task claim time:

Token family	Example	Source
Scope variable	<code>\${var.chromosome}</code>	Scope chain walk
Indexed array in scope	<code>\${var.orderedComparisonLabels[0]}</code>	JSON array element
FOREACH context	<code>\${ctx.item},</code> <code>\${ctx.index}</code>	Current iteration
Loop context	<code>\${ctx.iterationNo},</code> <code>\${ctx.parallelIndex}</code>	<code>execution_context</code>
Prior task	<code>\${ctx.task.detect.result_code}</code>	Most terminal execution of <code>detect</code>

The resolver does **not** evaluate arithmetic or boolean expressions in placeholders. Any logic beyond simple lookup must run in a worker (or in the planner before the instance starts) and **publish** a scalar or JSON value into scope.

Control-flow reads: IF, SWITCH, and WHILE nodes specify `condition_var` / `switch_var`. The engine reads an **integer** from scope (0 = false / exit loop; non-zero = true / continue). If the variable is absent, it falls back to `condition_ref_node_key` / `switch_ref_node_key` (prior ACTION `result_code`). See [WORKFLOW_ENGINE_DELPHI.md](#) and [wf_sql_branch_parity.sql](#).

5.5 Worker-driven conditions and branching

For conditions that depend on **data-dependent computation**, the workflow pattern is:

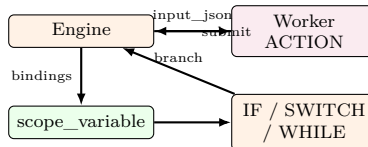


Figure 7: Worker-driven conditions: ACTION output updates scope, then control flow branches.

Example tree (from Delphi reference seed `DelphiTreeFlow`):

- `GateIf` — `condition_var = shouldRunQc`: worker sets 1 to run QC, 0 to skip.
- `ModeSwitch` — `switch_var = mode`: worker chooses normalization branch (1, 2, or default).
- `PollWhile` — `condition_var = hasMorePages`: pagination worker sets 1 until no pages remain, then 0.

This keeps the **workflow graph static** while allowing arbitrarily complex predicates inside workers. Bindings can take either:

- `source_kind = result_code` — worker returns a signed integer directly.
- `source_kind = output_path` — worker returns JSON; engine extracts a field (e.g. `$.gate.passed`) into scope.

5.6 Monte Carlo runs and stratified partitions

Monte Carlo validation ([packages/methylvalidation](#)) repeats the full pipeline on many **independent random splits** of the same cohorts. Each iteration:

1. Draws a **stratified partition** per cohort: fraction `train_fraction` (e.g. 0.8) for training, remainder for validation.
2. Shuffles samples **within each cohort** independently (binary: control + disease; multiclass: K cohorts).
3. Materializes a **run-specific** `project.json` (train sample lists, validation CSVs, output under `monte_carlo_runs/run_XXXX/`).
4. Runs centroid → detector → ... on that partition only.

Stratified split implementation: [methyl_validation/split.py](#) (`stratified_split`, `stratified_split_multiclass`).
Planner: [methyl_validation/planner.py](#).

5.6.1 Workflow engine bridge

Monte Carlo is represented explicitly in the DB, not only as filesystem artifacts:

Table / object	Content
<code>wf.monte_carlo_plan</code>	One row per instance: <code>base_project_path</code> , <code>layout</code> , <code>seed</code> , <code>feature_iterations</code> , <code>quality_iterations</code> , <code>config_json</code>
<code>wf.monte_carlo_run</code>	One row per planned iteration: <code>run_id</code> , <code>phase_name</code> (<code>feature</code> <code>quality</code>), <code>task_config_json</code>
Instance <code>context_json.monteCarlo</code>	Seeds plan metadata when the middle-tier starts an instance

Scoped variables for MC (instance root and per-iteration):

Variable	Meaning
<code>mc.enabled</code> , <code>mc.seed</code> , <code>mc.layout</code>	Plan metadata from <code>context_json</code>
<code>mc.featureIterations</code> , <code>mc.qualityIterations</code>	Loop bounds
<code>mc.phase</code> , <code>mc.phaseIteration</code> , <code>mc.runId</code>	Current iteration identity
<code>mc.taskConfig</code>	JSON descriptor for this run (project path, train/val lists, detector overrides)

On ACTION activation, the engine **injects** `mc.taskConfig` into worker `input_json` (top-level payload or `mcTaskConfig` property) so centroid/detector workers receive the stratified partition for **that** iteration without re-rolling the split.

Schema migration: [wf_monte_carlo_support.sql](#).

5.6.2 Example workflow topology (MethylValidationFlow)

Seed: [workflow_methylvalidation_seed.sql](#)

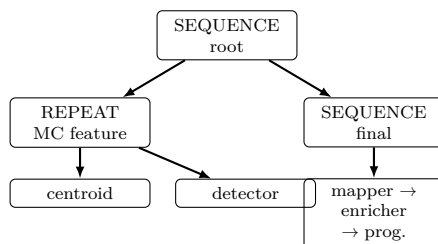


Figure 8: MethylValidationFlow: Monte Carlo feature loop then final full-sample pipeline.

Each **REPEAT** body receives a distinct `mc.taskConfig` describing one stratified draw. The **final sequence** runs once on the full sample set after all feature-stability iterations complete.

Future composition: outer REPEAT (or FOREACH over `wf.monte_carlo_run` rows) $\times$ inner FOREACH over chromosomes/comparisons — see [wf_foreach_design.md](#).

Example `context_json` fragment for MC instances:

```
{
  "monteCarlo": {
    "baseProject": "/work/study/project.json",
    "layout": "binary",
    "seed": 42,
    "featureIterations": 30,
    "qualityIterations": 20
  }
}
```

Project-level MC parameters also live under `step_config.validation` in the portal schema (`validation_monte_carlo` in [validation.schema.json](#)): `train_fraction`, `n_iterations`, cohort CSVs, and regulatory metadata.

5.7 Cluster metadata in DB

```
wf.cluster (
  cluster_key, name, provider,
  shared_storage_uri,    -- e.g. file://share or https://...
  worker_mount_path,     -- default '/work'
  status
)
```

Workers register against a cluster; capabilities are stored on `wf.worker.capabilities` (JSON).

6 5. Two-group example: config → workflow tree

This section ties **one control vs two disease groups** (OvR-style) to the **DataDrivenPipeline** workflow definition and a concrete `context_json`.

6.1 5.1 Project intent (PCa3-style)

Item	Value
Control	<code>controls.all</code>
Disease stages	e.g. <code>PCa_Low</code> , <code>PCa_High</code> (two comparisons)
Context	<code>CG</code>
Chromosomes	24 (1–22, X, Y)
Comparisons shorthand	<code>control_vs_each_disease</code>

Resolved directories (under `/work/prostate-cancer/PCa3/`):

- Centroids control: `centroids/controls/healthy/all`
- Centroids disease: `centroids/diseases/cancer/{PCa_Low|PCa_High}`
- Detections: `detections/all/{PCa_Low|PCa_High}`

6.2 5.2 Instance `context_json` (runtime)

From `workflow_engine/sql/instance_context_examples/pca_ovr.json`:

```
{
  "projectPath": "/home/ubuntu/Work/prostate-cancer/configs/project_PCa3.json",
  "context": "CG",
  "centroid1Dir": "/work/prostate-cancer/PCa3/centroids/controls/healthy/all",
  "group1Label": "group1",
  "orderedComparisonLabels": ["PCa_Low", "PCa_High"],
  "chromosomes": ["1", "2", "...", "22", "X", "Y"],
  "comparisons": [
    {
      "label": "PCa_Low",
      "centroid2Dir": "/work/prostate-cancer/PCa3/centroids/diseases/cancer/PCa_Low",
      "detectOutDir": "/work/prostate-cancer/PCa3/detections/all/PCa_Low",
      "group2Label": "group2"
    },
    {
      "label": "PCa_High",
      "centroid2Dir": "/work/prostate-cancer/PCa3/centroids/diseases/cancer/PCa_High",
      "detectOutDir": "/work/prostate-cancer/PCa3/detections/all/PCa_High",
      "group2Label": "group2"
    }
  ]
}
```

```

]
}

```

No per-chromosome nodes are stored in the DB — only **~15 static nodes** in `DataDrivenPipeline`; fan-out is entirely data-driven.

6.3 5.3 Workflow tree (`DataDrivenPipeline`)

Seed: [workflow_engine/sql/wf_data_driven_pipeline_seed.sql](#)

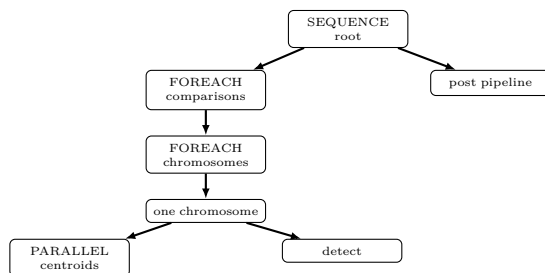


Figure 9: `DataDrivenPipeline` workflow tree (static nodes, data-driven fan-out).

6.4 5.4 Per-chromosome dependency (one comparison)

```

one_chromosome (SEQUENCE)
  PARALLEL centroids
    centroid_g1 (methyl-centroid → centroid1Dir)
    centroid_g2 (methyl-centroid → centroid2Dir)
  detect       (methyl-detector → detectOutDir)

```

`detect` runs only after **both** centroids for that chromosome succeed (PARALLEL parent must complete first).

6.5 5.5 Execution scale (PCa OvR example)

Phase	Count
Comparisons (parallel)	2
Chromosomes per comparison	24
Actions per chromosome	3 (2 centroids + 1 detect)
Subtotal ACTION tasks	2 × 24 × 3 = 144
Post-pipeline (sequential)	mapper + enricher + progression = 3
Total ACTION executions	147

Post steps run only after **all** comparison branches complete (root `SEQUENCE`: fan-out then `post_pipeline`).

6.6 5.6 Input templates (resolved at claim time)

Node	Resolved input_json fields
centroid_g1	project, group, chromosome, context, comparison, outputDir → centroid1Dir
centroid_g2	same with centroid2Dir
detect	project, chromosome, centroid1Dir, centroid2Dir, outputDir
mapper / enricher	project only (reads detection outputs from shared storage)
progression	project, orderedComparisonLabels

Worker contracts: [workflow_engine/sql/wf_worker_contracts_pca_ovr.md](#)

7 6. Worker protocol and middle-tier

Workers **never connect to the database directly**. They use the REST API (or an equivalent gateway) documented in [contracts/openapi.yaml](#) and [workflow_engine/WORKER_PROTOCOL.md](#).

7.1 REST endpoints

Method	Path	Caller
POST	/v1/workers/authenticate	Worker registration
POST	/v1/workers/tasks/request	Worker poll loop
POST	/v1/workers/tasks/{id}/submit	Task completion
POST	/v1/workers/tasks/{id}/heartbeat	Long-running jobs
POST	/v1/workers/tasks/{id}/fail	Explicit failure
POST	/v1/workflows/instances	Portal / admin
GET	/v1/workflows/instances/{id}	Portal status

Implementation: [workflow_engine/src/WfEngine.RestApi.pas](#), served by [WfEngine.RestHttpServer.pas](#) (default port **8080**).

Python gateway (PostgreSQL parity testing): [workflow_engine/rest/gateway.py](#).

7.2 Request / submit sequence

7.3 Middle-tier responsibilities



Figure 10: Worker poll / submit protocol via stateless middle-tier.

Component	Role
<code>WfEngine.RestHttpServer</code>	HTTP →
<code>TRestApiService</code>	<code>TRestApiService.Handle</code> Auth, JSON parse, call <code>sp_*</code> or Delphi engine path
<code>TWorkflowEngine</code> / <code>SQL procs</code>	Activate graph, resolve templates, advance control flow
<code>TWorkflowEngineHostedService.RunUntilStopped</code>	Observability tick only — does not dispatch tasks

State: All workflow progress lives in the database (`node_execution.status`, `task_lease`, `scope_variable`). The middle-tier is **stateless** between HTTP calls.

7.4 Capability matching

workflow_action	Capability string	Worker advertises
<code>pipeline.centroid</code>	<code>methyl-centroid</code>	<code>methyl-centroid</code>
<code>pipeline.detector</code>	<code>methyl-detector</code>	<code>methyl-detector</code>
<code>pipeline.mapper</code>	<code>methyl-mapper</code>	<code>methyl-mapper</code>
<code>pipeline.enricher</code>	<code>methyl-enricher</code>	<code>methyl-enricher</code>
<code>pipeline.progression</code>	<code>methyl-disease- progression</code>	<code>methyl-disease- progression</code>

`sp_worker_request_task` claims the oldest `READY` row matching the worker's capability (optional filter), using `FOR UPDATE SKIP LOCKED` on PostgreSQL for safe concurrent polling.

8 7. Cluster and shared storage model

Distributed processing assumes **all worker nodes see the same filesystem** at a common mount point, conventionally `/work`, recorded in `wf.cluster.worker_mount_path` and optionally `shared_storage_uri`.

8.1 Assumptions

1. **Shared storage:** NFS, Azure Files, or equivalent mounted at `/work` on every compute node.
2. **Path contract:** `input_json` carries absolute paths under that mount (`centroid1Dir`, `detectOutDir`, ...). Workers read and write files directly; the DB stores paths, not file bytes.
3. **No worker-to-worker RPC:** Dependencies are enforced only by the workflow tree (e.g. `detect` after both centroids for the same chromosome).
4. **Maximize parallelism:** Outer `FOREACH` over comparisons and inner `FOREACH` over chromosomes both use `foreach_parallel = 1`, so up to `len(comparisons) × len(chromosomes)` centroid pairs and detections can run concurrently, bounded by worker count.
5. **Serialization within a chromosome:** For each (`comparison`, `chromosome`), `detect` waits for both centroids — the only mandatory per-chromosome barrier.
6. **Post-pipeline barrier:** Mapper, enricher, and progression run only after **all** detection tasks complete (second child of root `SEQUENCE`).

8.2 Architecture diagram

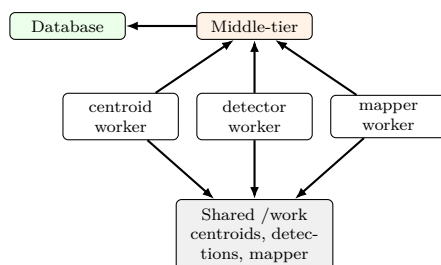


Figure 11: Cluster workers share NFS/Azure Files storage; coordination via database only.

8.3 Minimizing total processing time

Technique	Effect
Per-chromosome parallelism	24 chromosomes per comparison run concurrently
Per-comparison parallelism	Multiple disease groups run concurrently
Capability-specific workers	Centroid-heavy and detector-heavy nodes scale independently
SKIP LOCKED task claim	Many workers poll without blocking each other
Shared output paths	Centroid output for chr <i>N</i> is input to detect for chr <i>N</i> on the same mount — no data shipping between nodes

8.4 Portal + planner + engine (end-to-end)

1. User edits `project.json` in portal (schema-validated)
2. Planner builds `context_json` (`comparisons[]`, `chromosomes[]`, `dirs`)
3. Portal POST `/v1/workflows/instances { workflow_version_id, context_json }`
4. Engine activates `DataDrivenPipeline` → thousands of `READY` rows materialized logically
5. Workers on cluster pull tasks, `read/write /work/...`, submit results
6. Instance status → `COMPLETED` when root `SEQUENCE` finishes

9 Further reading

Document	Topic
workflow_engine/sql/DataDrivenPipeline.md	FOREACH workflow, deploy order
workflow_engine/WORKFLOW_ENGINE_DELPHI.md	Scope variables, IF/SWITCH/WHILE, Monte Carlo bridge
workflow_engine/CAPABILITY_CHECK.md	Engine capabilities vs gaps
docs/theory/chapters/05-methylpredictor-and-validation.qmd	Theory: Monte Carlo splits and validation
docs/theory/chapters/11-project-configuration.qmd	Theory: project configuration
tools/methyl-config-editor/README.md	Config editor setup
workflow_engine/README.md	SQL deploy order

Document version: aligns with `DataDrivenPipeline` seed, `FOREACH` engine support, scoped-variable write-path parity, and Monte Carlo plan/run tables.

10 Rendering this document

This Quarto source is generated from [pipeline_architecture.md](#). Edit the Markdown, then regenerate:

```
python workflow_engine/docs/build_pipeline_architecture_qmd.py
quarto render workflow_engine/docs/pipeline_architecture.qmd --to html
quarto render workflow_engine/docs/pipeline_architecture.qmd --to pdf
```

- **HTML** uses interactive Mermaid diagrams.
- **PDF** uses TikZ vector figures (same approach as [docs/MethylPipeline-overview.qmd](#)).
- PDF requires a TeX installation (`quarto install tinytex` or TeX Live with `lualatex`).